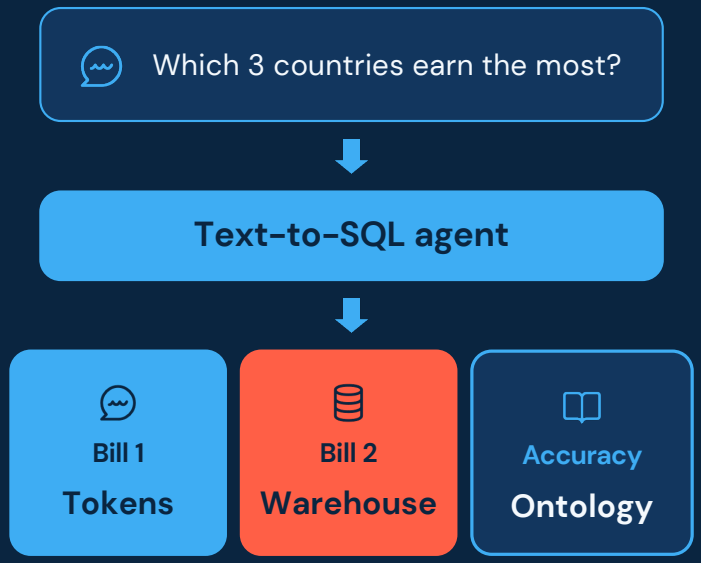
TorontoAI × AWS User Group Toronto

Two bills and one ontology

A database utility for text-to-SQL on a lakehouse: measure the token bill, the warehouse bill and the accuracy, then improve all three

Chandan Kumar, BeCloudReady, Databricks Registered Partner



Each era changed what we pay for



1980s on
On-prem warehouse

Pay for capacity

Big box, storage tied to compute

2012 on
Cloud warehouse

Pay per use

Elastic, so cost work starts

2010s
Data lake

Pay for storage

Cheap files, weak governance

2020s
Lakehouse

Pay for compute

Open tables, one governed copy

+ pay per token

How companies use AI today

Business and operations	**Microsoft Copilot**	Licence or credits, no code
Developers	**GitHub Copilot**	Many models, usage controls
Special use cases	**Claude**	Few, high value
Internal apps	**AskXX web tool**	Streamlit or Open WebUI on Bedrock, other APIs, on-prem
Inside Databricks	**Genie**	Stays on its platform
Inside Snowflake	**Cortex**	Stays on its platform

THE CHALLENGE

6 tools

Each one needs its own prompts, skill files and context.

What your data means gets copied six times, then drifts: 'Y' is approved, revenue is SUM(totalPrice).

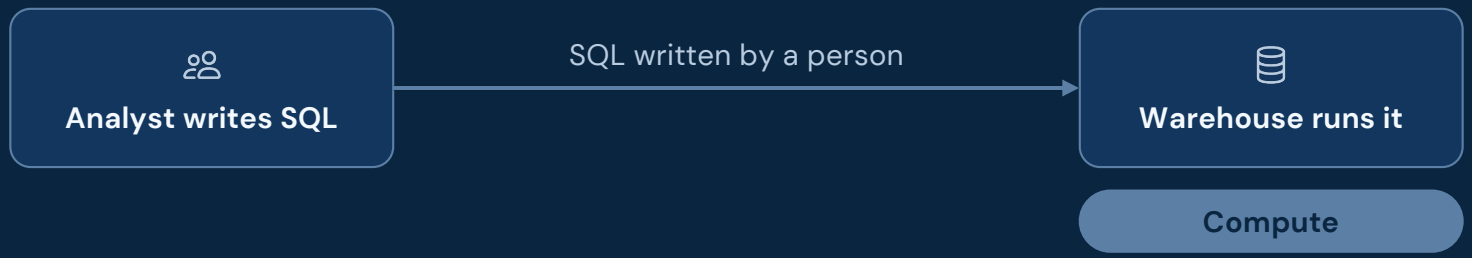
General tools: they reach data from outside

Platform tools: they live inside one data platform

Then one bill. Now two bills per question.

BEFORE

1 bill



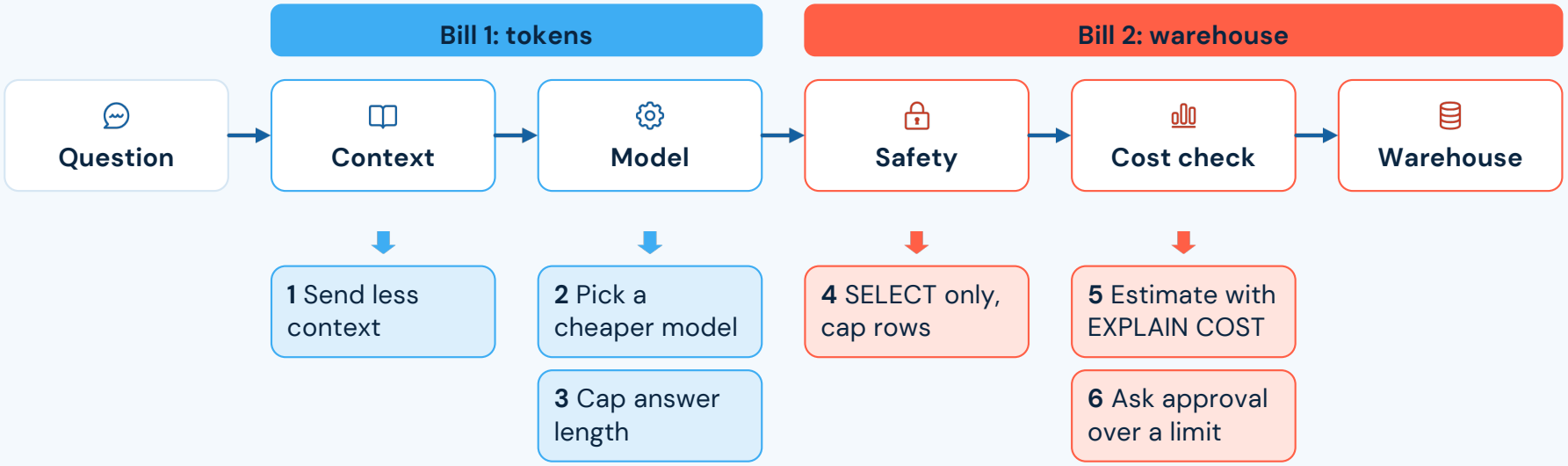
NOW

2 bills



A chat box turns every employee into a query writer. Volume goes up and **both meters run**.

Six levers, each tied to one bill



Which of these can your current tool change today?

db-agent: one question, six visible steps



STEPS 1 AND 2: WHAT GOES IN

"Which 3 countries earned the most net revenue?"

Schema sent to the model

```
retail_gold.daily_sales
country, net_revenue, orders
```

STEP 3, BILL 1: SQL WRITTEN

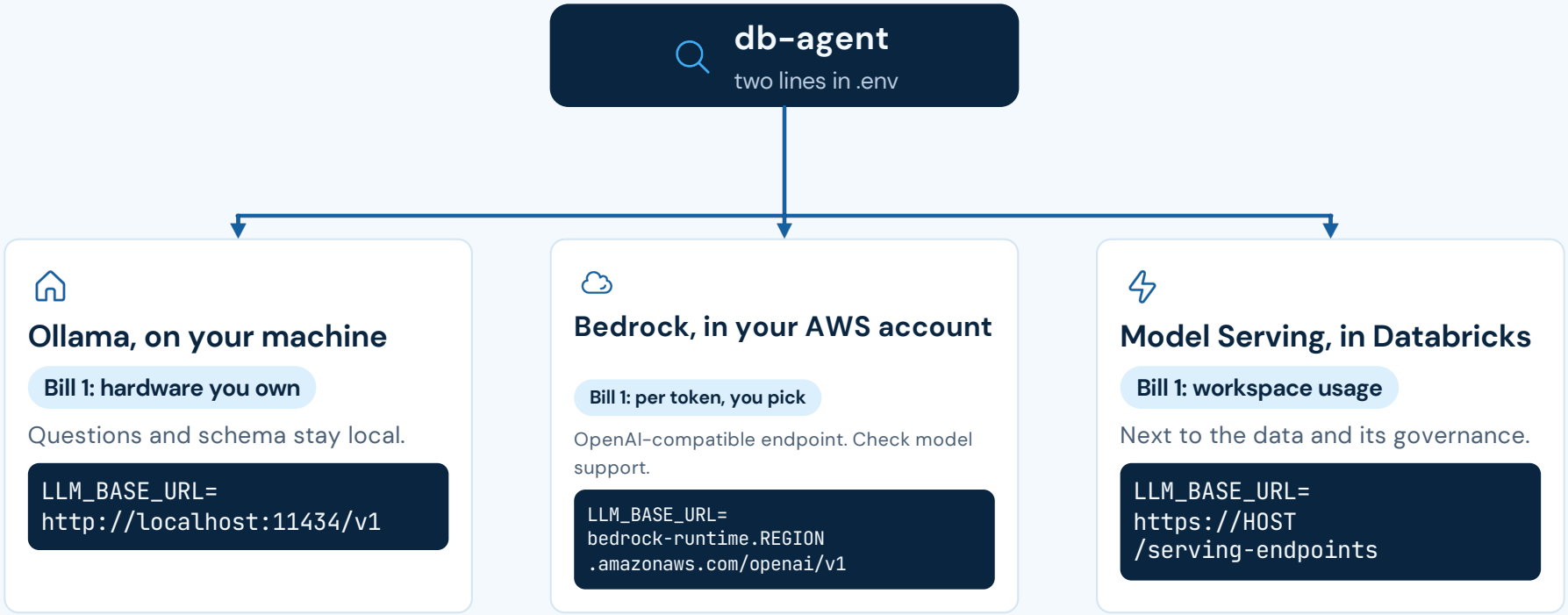
```
SELECT country,
SUM(net_revenue) AS rev
FROM retail_gold.daily_sales
GROUP BY country
ORDER BY rev DESC LIMIT 3
```

Illustrative, not a recorded run

STEPS 4 TO 6, BILL 2: CHECKS

- ✓ SELECT only: passes
- ✓ EXPLAIN COST: estimate shown
- ⚠ Over the limit: ask first

Where the model runs decides Bill 1



Most wrong answers are facts about your data

QUESTION	WITHOUT AN ONTOLOGY	WITH ONE
How many suppliers are approved?	`approved = 'Yes'` returns 0	`approved = 'Y'` returns 27
How many transactions were paid with visa?	`paymentMethod = 'Visa'` returns 0	`paymentMethod = 'visa'` returns 1,332,880
How many customers live in the United States?	`country = 'United States'` returns 0	`country = 'USA'` returns 99
What is the revenue from female customers?	`gender = 'F'` returns null	`gender = 'female'` returns 35,881,278

WHAT AN ONTOLOGY IS

A small file of one-line notes: what values mean, how tables join, how revenue is defined.

WHAT A SCAN CAN DO

Reads metadata only and lists what is missing. A person writes the meaning.

Same 7B model, two containers: 14/17 became 17/17

PORT 3001, NO ONTOLOGY

14 /17 82%

Schema only

qwen2.5-coder:7b in Ollama

PORT 3002, WITH ONTOLOGY

17 /17 100%

Schema plus ontology

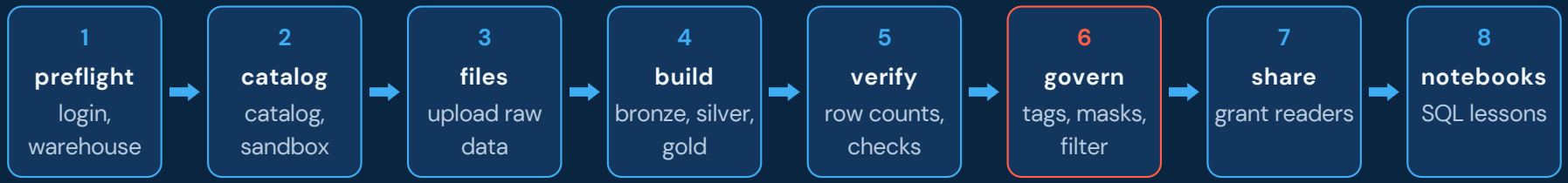
the same model, same questions

How many suppliers are approved?	**3001** 'Yes' returns 0	**3002** 'Y' returns 27
How many transactions were paid with visa?	**3001** 'Visa' returns 0	**3002** 'visa' returns 1,332,880
What is the revenue from female customers?	**3001** 'F' returns null	**3002** 'female' returns 35,881,278

Databricks Bakehouse data, 17 questions, one run each. Only the knowledge file differs. The ontology adds about 520 tokens a question, free on a local model.

labagent: a working lakehouse from one command

```
$ labagent launch dbx-lakehouse --name awsug
$ labagent destroy dbx-lakehouse --name awsug
```



2
domains: retail, healthcare

40
tables built and checked

1,067,371
real invoice lines

1
command to remove it all

Ask questions of gold, not bronze



Fewer, clearer tables mean fewer tokens in the prompt and smaller scans in the warehouse.

Live demo: six stops, 11 minutes

1

Ask

Revenue per product. The SQL, the rows, and both bills.

2 min

2

See it fail

No documentation: 0 rows, no error.

2 min

3

Scan

Metadata only. Readiness 40 of 100.

1.5 min

4

Write notes

Three one-line notes. Same questions, right answers.

2 min

5

Measure

Accuracy test on 17 questions, 3001 against 3002.

2 min

6

Ship

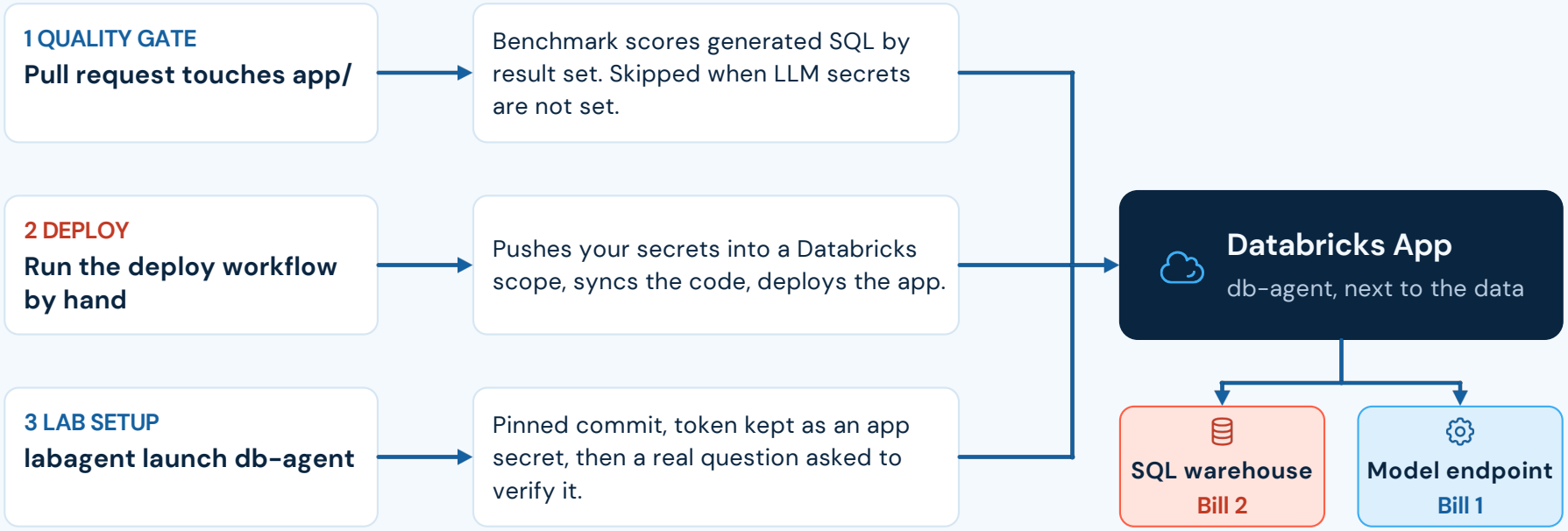
Open it as a Databricks App.

1.5 min

Use the app · See it fail · Fix and measure · Platform

Block widths match the minutes. Fallback for every stop: a recorded clip.

Same agent, three ways to ship it



Or run it on a laptop: one script or Docker Compose. The model is local, the warehouse is remote.

Any model, any database, one ontology

WHERE IT RUNS

Your laptop
One script or Docker Compose

AWS
Container next to your data

Databricks App
CI or labagent deploy

Snowflake
Not tested, a target to prove

THE UTILITY

db-agent

Try any model
Swap it with two lines in .env

See both bills
Token and warehouse estimates on every answer

Score accuracy
Benchmark questions, with and without the ontology

Share one ontology
One file for the web app, the API and MCP clients

WHAT IT TALKS TO

Ollama
Small local model

Bedrock or any API
OpenAI-compatible

Databricks SQL
Warehouse

Postgres, Lakebase
Direct

SQLite, DuckDB on MinIO
Files and local

Snowflake engine
Not built yet

Use it to experiment with models and queries, and as the daily agent for your own database tasks. github.com/db-agent/db-agent